# Individual Project

## CryptoChain Analyzer Dashboard

Hash Functions and Blockchain

Cryptography | Universidad Alfonso X el Sabio

Prof. Jorge Calvo — Academic Year 2025–26

---

**Project Summary**

Build a **real-time Python dashboard** that connects to a public blockchain API and displays live cryptographic metrics from the Bitcoin network. You must also integrate an **AI component** that adds analytical value to the raw blockchain data.

| | |
|---|---|
| **Format:** | Individual work |
| **Kick-off:** | 20 April 2026 (in-class session) |
| **Checkpoint:** | 29 April 2026 (in-class session — progress visible in GitHub) |
| **Deadline:** | **14 May 2026, 23:59** |
| **Delivery:** | GitHub Classroom repository link |
| **Grade:** | 0 to 4 points |

# 1 Session 1 Kick-off (20 April — 90 min)

This first session is a **working session**, not a lecture. By the end of it you must have completed three concrete milestones.

**Milestone 1 · GitHub Setup** (0–25 min)

1. Open the **GitHub Classroom invitation link** shared by the professor.
2. Click **Accept assignment** with your own GitHub account. Your private repository will be created automatically from the class template. Do not create a different repository by yourself.
3. Open your repository and complete the **README.md**. It does not need to be perfect — include:
   - Your name and GitHub username.
   - A short project title.
   - Which AI approach from M4 you think you will use.
   - The initial status of Modules M1–M4.
   - One short sentence in "Next Step".
4. Make your **first commit** with this updated README.

## Milestone 2 · First API Call (25–65 min)

Write a short Python script — **10 lines maximum** — that connects to one of the recommended APIs and prints real Bitcoin data to the console.

To get you started with the syntax of the `requests` library:

```
import requests

url = "https://blockstream.info/api/..."
# choose an endpoint
response = requests.get(url)
data = response.json()
print(data)
```

**Your task:** explore the API documentation, find the right endpoint, and print at least the following fields for the latest block: block height, hash, difficulty, nonce, and number of transactions.

### Important 1: Connect the data to the theory

Look at the values you get. Can you see the leading zeros in the block hash? Can you relate the `bits` field to the target threshold from the notes? Write your observations as comments in the script.

## Milestone 3 · First Code Commit (65–80 min)

Push your work to GitHub before the session ends:

1. Clone **your GitHub Classroom repository** on your machine.
2. Keep the template structure. Do not delete the main files.
3. Place your first API script inside `api/blockchain_client.py` or inside the M1 module if you prefer.
4. Update the README status if needed.
5. Commit and push everything:

```
git add .
git commit -m "Session 1: first API call, project structure"
git push
```

**Goal:** leave this session with at least **two commits** visible in your GitHub Classroom repository and real Bitcoin data on your screen.

# 2 Learning Objectives

By completing this project you will:

- Apply the cryptographic concepts studied in Topic 7 (SHA-256, Proof of Work, Merkle trees, difficulty) to **real, live data**.
- Develop the ability to **read technical documentation** (API docs, whitepapers) and extract the information you need.
- Integrate an **AI model** into a data pipeline and evaluate its performance.
- Manage a software project using **GitHub** from the first day.

# 3 What to Build

Your dashboard must have a **required core** (Modules 1–4) and may include **optional extensions** (Modules 5–7) for a higher grade.

## 3.1 Required Modules (M1–M4)

**M1 · Proof of Work Monitor**

Show live data about the current state of Bitcoin mining:

- Current difficulty value and its visual representation as a **leading-zero threshold** in the 256-bit SHA-256 space.
- Time between the last $N$ blocks (plot the distribution — what statistical distribution do you expect? why?).
- Estimated current network hash rate.

*Hint: review Section 6 of the Blockchain notes. The field* `bits` *in the block header encodes the target.*

### M2 · Block Header Analyzer

Display the 80-byte structure of the latest block header and verify the Proof of Work locally:

- Show all six fields of the header (version, prev_hash, merkle_root, timestamp, bits, nonce).
- Use Python's `hashlib` to compute `SHA256(SHA256(header))` and confirm that the result is below the target. **Show this verification step clearly.**
- Count the number of leading zero bits in the block hash.

*Hint: pay attention to byte order (little-endian) when parsing the header fields.*

### M3 · Difficulty History

Plot the evolution of the difficulty over the last several adjustment periods (one period = 2016 blocks):

- Show the difficulty value over time.
- Mark each **difficulty adjustment** event on the chart.
- Show the ratio between actual block time and the target (600 seconds) for each period.

*Hint: the adjustment formula is in Section 6.1 of the notes.*

### M4 · AI Component (choose one)

Implement **at least one** of the following AI approaches and **evaluate it with appropriate metrics**:

1. **Predictor:** Predict the next difficulty adjustment value using a time-series model (e.g. Prophet, LSTM, or any regression model you justify).
2. **Anomaly detector:** Identify blocks whose inter-arrival time is statistically abnormal. An exponential distribution is the expected baseline — deviations may indicate mining pool behaviour.
3. **Fee estimator:** Predict the optimal transaction fee (sat/vByte) from mempool data using a supervised model.

**Required in the report:** state which model you chose, why, what data you used to train it, and the evaluation metric(s).

### 3.2 Optional Modules (M5–M7)

These modules are not required but allow you to reach the highest grade levels. Choose any combination.

| Module | Title | Description |
|---|---|---|
| M5 | Merkle Proof Verifier | Pick any transaction in a block and verify its Merkle proof step by step. Show each hash computation. |
| M6 | Security Score | Estimate the cost (in USD/hour) of a 51% attack on Bitcoin based on live hash rate data. Visualise how confirmation depth reduces attack probability (Nakamoto 2008, §11). |
| M7 | Second AI approach | Implement a second AI method (different from the one in M4) and compare results. |

## 4 Recommended APIs

The following free APIs require no registration or API key:

| API | URL | Notes |
|---|---|---|
| Blockstream | `blockstream.info/api` | Block data, headers, transactions |
| Mempool.space | `mempool.space/api` | Mempool, fees, real-time WebSocket |
| Blockchain.info | `blockchain.info/api` | Aggregated stats, difficulty history |

**Important 2: Read the documentation**

Before writing any code, read the API documentation. Understand what each endpoint returns and what the fields mean. Do not assume that field names are self-explanatory.

## 5 Dashboard Framework

You must build an **interactive visual dashboard** — not a command-line script. Two Python frameworks are suitable for this project:

- **Streamlit** — `streamlit.io` Simple to start with. Recommended if you have no prior experience with web dashboards.
- **Dash** (Plotly) — `dash.plotly.com` More flexible and professional. Recommended if you are comfortable with Python callbacks.

Both have extensive documentation and tutorials. **Researching how to use them is part of the project.** The dashboard must update automatically — do not require the user to refresh the page manually.

# 6 GitHub Requirements

**GitHub is mandatory from Day 1**

The project must live in a GitHub repository **from the first day of work**. Commit history is part of the evaluation — a repository with all commits in the last 48 hours before the deadline **will receive a penalty in Criterion 5**.

## 6.1 Repository Setup (do this today)

1. Create a GitHub account if you do not have one yet.
2. Open the **GitHub Classroom invitation link** shared by the professor.
3. Click **Accept assignment**.
4. Wait until GitHub Classroom creates your private repository.
5. Open the repository created for you and **do not rename it**.
6. Use that repository for all your work during the project.

## 6.2 Required Repository Structure

```
cryptochain-dashboard-yourname/
README.md          # Project description + how to run
requirements.txt    # All Python dependencies
app.py              # Dashboard entry point
modules/            # One file per module (M1, M2...)
api/                # API connection code
report/            # Final PDF report (add before deadline)
```

**Important 3: Use the repository created by Classroom**

You do not need to create a repository manually. GitHub Classroom creates the private repository for you from the course template. Use that repository and keep the main files and folders.

## 6.3 README: what you must update

The **README.md** is required and will be used by the professor to follow your progress during the project. Update it every week.

- **Student Information** — your name and GitHub username
- **Project Title** — a short title for your dashboard
- **Chosen AI Approach** — the AI option you plan to implement
- **Module Tracking** — status of M1, M2, M3, and M4
- **Current Progress** — 3 to 5 short lines about what you have done
- **Next Step** — the next small action you will do
- **Main Problem or Blocker** — what is stopping you right now

**Important 4: Keep the README honest and updated**

Do not write what you plan to have done in the future as if it were already complete. The README must reflect the real state of your project.

**Important:** in GitHub Classroom you will receive a simplified template. The main files are: `README.md`, `requirements.txt`, `app.py`, `api/blockchain_client.py`, and the files inside `modules/`.

## 6.4 Quick Git Reference

**First time on your machine:**

```
git clone https://github.com/.../your-classroom-repository.git
cd your-classroom-repository
```

**Daily workflow:**

```
git pull                              # Get latest changes
...  work on your code ...
git add .                             # Stage all changes
git commit -m "Implement M1:  block time histogram"   # Save with message
git push                              # Upload to GitHub
```

**Write meaningful commit messages.** Describe *what* you did and *why*, not just "changes" or "fix".

### 6.5 Expected Commit Timeline

| By date | What should be visible in GitHub |
|---|---|
| 21 April | GitHub Classroom repository accepted. README completed. API connection returning real data (even if just printed to console). |
| 28 April | M1 and M2 working and visible in the dashboard. M3 started. AI approach decided and documented in README. |
| 29 April | **CHECKPOINT (in-class):** M1 + M2 + M3 operational. At least a skeleton of the AI module present. |
| 7 May | M4 (AI) complete and integrated. Optional modules if attempted. |
| 14 May | Final version. README complete. Final report added to repo. |

## 7 Deliverables

Submit **one link** to your GitHub Classroom repository by **14 May 2026, 23:59**.

The repository must contain:

- **Working code** — the dashboard must run with `pip install -r requirements.txt` followed by a single command (e.g. `streamlit run app.py`).
- **README.md** — your name, GitHub username, chosen AI approach, module status, current progress, next step, and how to run.
- **Final report in PDF** — 2–3 pages explaining: (1) the cryptographic metrics you display and their meaning, (2) the AI model chosen, why, and its evaluation results, (3) at least one external reference (academic paper, whitepaper, or technical documentation). Add this PDF to the repository before the deadline.

**Important 5: Academic integrity**

The use of AI tools (ChatGPT, Copilot, etc.) to generate code or the report is **allowed as a support tool only**. You must understand every line of code you submit and be able to explain any part of it. Evidence of copy-paste without understanding will result in a grade of 0. The commit history must reflect **your own progressive work**.

## 8 Grading Rubric

The project is graded from **0 to 4 points**.

$$\text{Grade} = 0.30 \cdot C_1 + 0.25 \cdot C_2 + 0.20 \cdot C_3 + 0.15 \cdot C_4 + 0.10 \cdot C_5$$

### C1 · Cryptographic Correctness (30%)

| | |
|---|---|
| **4** | All metrics are correct and connected to theory from Topic 7. The student verifies at least one block hash manually using `hashlib` and explains what the `bits` field represents as a 256-bit threshold. |
| **3** | Core metrics are correct (difficulty, block time, Merkle root). Minor errors in derived values (e.g. hash rate estimate). |
| **2** | Data is fetched and displayed but with conceptual inaccuracies. No manual verification of results. |
| **1** | Serious conceptual errors (e.g. nonce confused with block hash, or `bits` field displayed without understanding). |
| **0** | No cryptographic metrics implemented, or values are hardcoded. |

### C2 · AI Component (25%)

| | |
|---|---|
| **4** | Model is well chosen and justified. Trained on real blockchain data. Evaluated with appropriate metrics (MAE, F1, AUC...). The report explains model choice and limitations. |
| **3** | Model works and produces coherent predictions. Basic evaluation present. Justification brief but correct. |
| **2** | Model runs but lacks evaluation, or is clearly unsuitable for the problem. |
| **1** | AI code present but not integrated into the dashboard, or copied from a tutorial without adaptation. |
| **0** | No AI component, or code that produces no results. |

### C3 · Functionality and Real-Time Updates (20%)

| | |
|---|---|
| **4** | Dashboard runs and updates automatically (polling $\leq$ 60s or WebSocket). Handles API errors without crashing. At least 3 modules operational. |
| **3** | Automatic updates work in main modules. Minor manual intervention needed elsewhere. Basic error handling present. |
| **2** | Dashboard runs and shows real data but requires manual refresh. Or only historical data shown (no real-time). |
| **1** | Dashboard starts but crashes frequently. Some data hardcoded. |
| **0** | Dashboard does not run, or shows only console output. |

### C4 · Visualisation (15%)

**4** Chart type matches the data (time series for difficulty, histogram for inter-block times, etc.). Axes labelled, scales correct, titles descriptive. Understandable without prior explanation.

**3** Charts correct with minor presentation issues (missing labels, automatic scale problems). Interface usable.

**2** Charts present but poorly chosen for the data type, or missing labels and titles.

**1** Only raw tables or text output. No interactive charts.

**0** No visualisation. Only `print()` output.

### C5 · Research and Report (10%)

**4** At least one metric goes beyond the course notes (e.g. nonce statistical distribution, mempool fee market, 51% attack cost). Report cites at least one academic or technical source (Nakamoto whitepaper, API docs, paper). Commit history shows consistent work over the full period.

**3** Some evidence of research with basic references. At least one new concept implemented correctly.

**2** Report describes the dashboard but adds nothing beyond class material. No external references.

**1** Minimal README only. No report. Dashboard does not go beyond exactly what was shown in class.

**0** No report, no README, or evidence of fully AI-generated work without understanding. All commits in the last 48h.

| **Example profile** | C1 | C2 | C3 | C4 | C5 | **Grade** |
|---|---|---|---|---|---|---|
| Excellent in all areas | 4 | 4 | 4 | 4 | 4 | **4.0** |
| Strong crypto, weak AI | 4 | 2 | 4 | 3 | 2 | **3.1** |
| Core only, no extras | 3 | 2 | 3 | 2 | 1 | **2.5** |
| Works but superficial | 2 | 1 | 3 | 2 | 1 | **2.0** |
| API connected, little else | 1 | 1 | 2 | 1 | 0 | **1.2** |

# 9 Frequently Asked Questions

**Can I use Bitcoin testnet instead of mainnet?**

Yes, but justify why in the report and note any differences in data availability.

**Can I analyse Ethereum or another blockchain instead?**

You may add a second blockchain for optional modules, but the required core (M1–M4) must use Bitcoin.

**What if the API is down during testing?**

Good software handles this gracefully. Design your code to catch API errors. Having a fallback to cached data is a plus, not a requirement.

**How many commits is "enough"?**

There is no minimum count. What matters is that the history shows *progressive, meaningful work* from day one. Four commits over four weeks is more credible than forty commits on the last night.

**Can I use any Python library?**

Yes. List every dependency in `requirements.txt`. If a library does the cryptographic work for you (e.g. a library that computes the Merkle root automatically), explain it in the report and also implement it manually so you demonstrate understanding.

---

Good luck. The goal is not to build a perfect product — it is to demonstrate that you can **apply cryptography to real data, investigate independently, and build something that works**.